DATABASE SCHEMA

PASSENGERS

```
CREATE TABLE passengers(
passenger_id NUMBER PRIMARY KEY,
name VARCHAR2(50), age NUMBER,
gender VARCHAR2(10),
phone VARCHAR2(15)
);
```

FLIGHTS

```
CREATE TABLE flights( flight_id
NUMBER PRIMARY KEY, airline
VARCHAR2(50), source
VARCHAR2(50), destination
VARCHAR2(50), departure_time
DATE, arrival_time DATE,
total_seats NUMBER
);
```

BOOKINGS

```
 CREATE TABLE bookings (
   booking_id NUMBER GENERATED
BY DEFAULT AS IDENTITY PRIMARY
KEY,

   passenger_id NUMBER NOT NULL,
   flight_id NUMBER NOT NULL,

   seat_number NUMBER NOT NULL,
```

```
   booking_date DATE DEFAULT
SYSDATE,

   status VARCHAR2(20) DEFAULT
'CONFIRMED',

   CONSTRAINT fk_passenger
      FOREIGN KEY (passenger_id)
      REFERENCES
passengers(passenger_id),

   CONSTRAINT fk_flight
      FOREIGN KEY (flight_id)
      REFERENCES flights(flight_id),

   CONSTRAINT
unique_seat_per_flight
      UNIQUE (flight_id,
seat_number)
);
```

PAYMENTS

```
CREATE TABLE payments( payment_id
NUMBER PRIMARY KEY, booking_id
NUMBER, amount NUMBER,
payment_method VARCHAR2(20),
payment_date DATE,

FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) );
```

Insert Sample Data Insert Passengers

```
INSERT INTO passengers VALUES (1,'Krish',21,'Male','9876543210');
INSERT INTO passengers VALUES (2,'Rahul',25,'Male','9876543211');
```

Insert Flights

```
INSERT INTO flights VALUES
(101,'Indigo','Delhi','Mumbai',SYSDATE,SYSDATE+1,120);
```

```
INSERT INTO flights VALUES
(102,'Air India','Delhi','Bangalore',SYSDATE,SYSDATE+1,150);
```

Create Sequence (Auto ID Generator)

```
CREATE SEQUENCE booking_seq
START WITH 1001
INCREMENT BY 1;

CREATE SEQUENCE payment_seq
START WITH 5001
INCREMENT BY 1;
```

STORED PROCEDURES

Procedure (Book Ticket)

```
CREATE OR REPLACE PROCEDURE book_ticket
(
p_passenger NUMBER, p_flight
NUMBER,
p_seat NUMBER
)
IS
BEGIN
INSERT INTO bookings
VALUES(booking_seq.NEXTVAL,p_passenger,p_flight,p_seat,SYSDATE);
END; /
```

Step 5 — Cancel Ticket Procedure

```
CREATE OR REPLACE PROCEDURE cancel_ticket
(
   p_booking NUMBER
```

```
)
IS
   v_count NUMBER;
BEGIN
   -- Check if booking exists
   SELECT COUNT(*) INTO v_count
   FROM bookings
   WHERE booking_id = p_booking;

   IF v_count = 0 THEN
      RAISE_APPLICATION_ERROR(-20001, 'Booking ID not found');
   END IF;

   -- Check if already cancelled
   SELECT COUNT(*) INTO v_count
   FROM bookings
   WHERE booking_id = p_booking
   AND status = 'CANCELLED';

   IF v_count > 0 THEN
      RAISE_APPLICATION_ERROR(-20002, 'Booking already cancelled');
   END IF;

   -- Update booking status instead of deleting
   UPDATE bookings
   SET status = 'CANCELLED'
   WHERE booking_id = p_booking;

EXCEPTION
   WHEN OTHERS THEN
      RAISE_APPLICATION_ERROR(-20003, 'Error cancelling ticket: ' ||
SQLERRM);
END;
/
```

Step 6 — Payment Procedure

```
CREATE OR REPLACE PROCEDURE make_payment
(
p_booking NUMBER, p_amount
NUMBER,
```

```
p_method VARCHAR2
)
IS
BEGIN
INSERT INTO payments
VALUES(payment_seq.NEXTVAL,p_booking,p_amount,p_method,SYSDATE);
END; /
```

Step 7 — Trigger (Prevent Overbooking)

```
CREATE OR REPLACE TRIGGER check_seats
BEFORE INSERT ON bookings
FOR EACH ROW DECLARE
seat_count NUMBER;
BEGIN
SELECT COUNT(*) INTO seat_count
FROM bookings
WHERE flight_id = :NEW.flight_id;

IF seat_count >= 120 THEN
RAISE_APPLICATION_ERROR(-20001,'No seats available');
END IF;
END; /
```

Step 8 — View

```
CREATE VIEW flight_schedule AS
SELECT flight_id, airline, source, destination
FROM flights;
```

Step 9 — Test Queries Show Flights

```
SELECT * FROM flights;
```

Book Ticket

```
EXEC book_ticket(1,101,12);
```

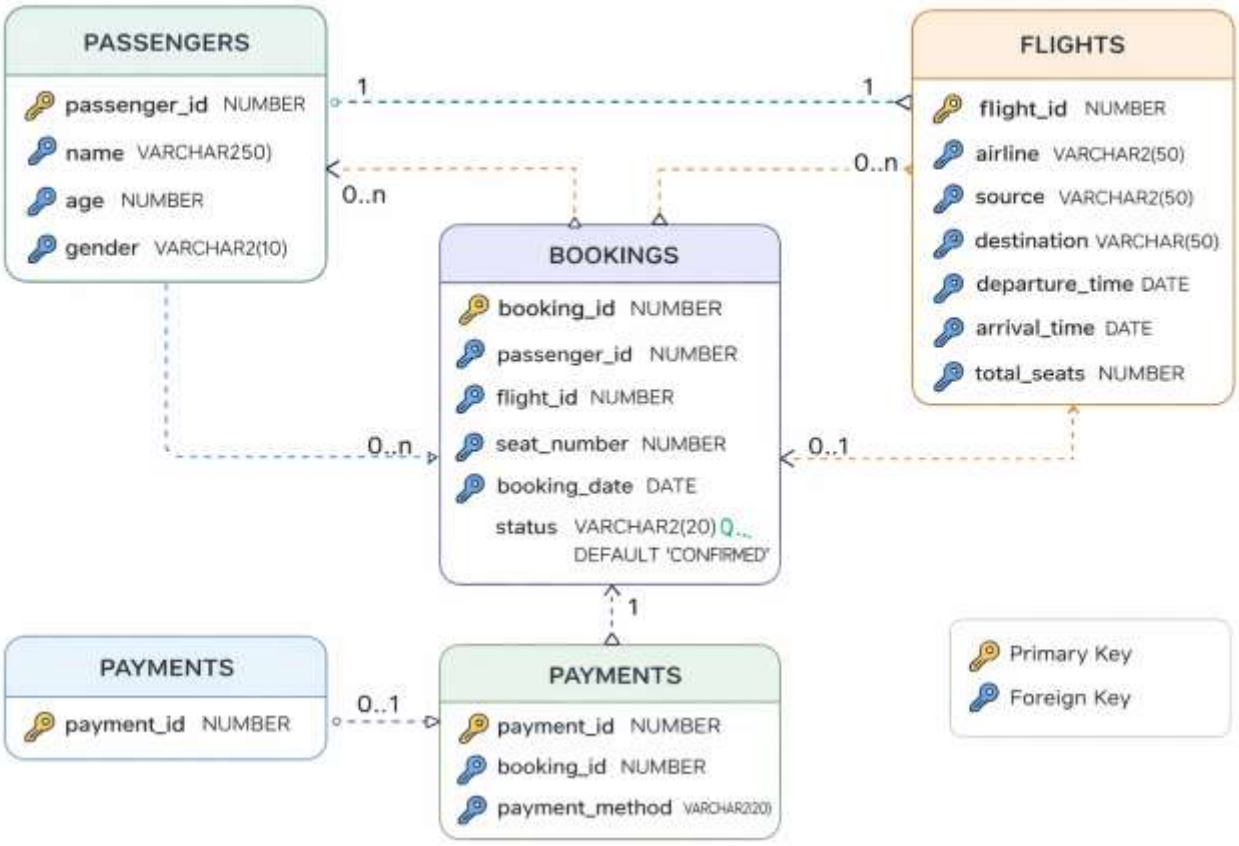
Make Payment

EXEC make_payment(1001,5000,'UPI');

Show Bookings

SELECT * FROM bookings;

Show Payments

SELECT * FROM payments;

ER-DIAGRAM